

Apache Kafka Architecture, Setup & Interview Guide: Taaskr Platform

1. Executive Summary & Introduction to Apache Kafka

What is Apache Kafka?

Apache Kafka is an open-source, highly throughput, distributed event-streaming platform designed to build real-time, event-driven data pipelines and asynchronous microservices architectures.

In a modern enterprise system like **Taaskr**, Kafka acts as the central **Event Bus** (or neural network). Rather than microservices communicating via direct, synchronous HTTP REST calls that block application threads and create tight coupling, services communicate asynchronously by publishing and consuming event messages.

Real-World Analogy for Beginners

Imagine a busy restaurant kitchen:

- **Synchronous (Without Kafka):** A waiter (Client) takes an order, walks to the kitchen, stands in front of the chef (Service), and waits silently doing nothing until the chef finishes cooking the meal. If 50 waiters stand waiting, the kitchen bottlenecks and freezes.
- **Asynchronous (With Kafka / Notice Board):** The waiter pins a slip ("Order #104: AC Repair") to a central, high-speed digital notice board (**Kafka Topic**). The waiter immediately returns to serve other customers. The kitchen chef (**Provider Service**), the cashier (**Payment Service**), and the SMS alert system (**Notification Service**) all independently read the slip off the board at their own pace and process it without delaying anyone else.

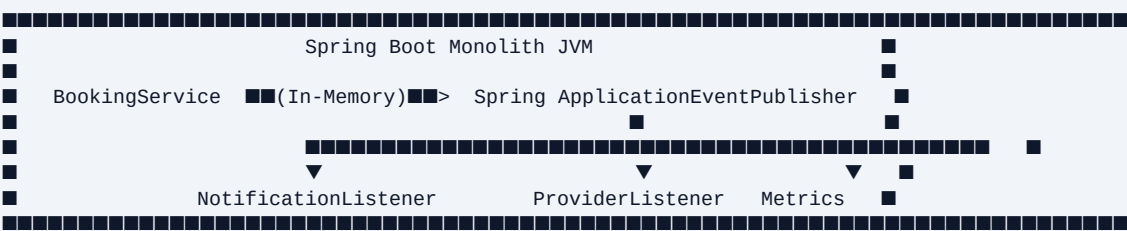
Key Kafka Concepts & Terminology

- **Producer:** A client application or microservice that writes (publishes) event data to Kafka (e.g., `BookingEngineService`).
- **Consumer:** A microservice that reads (subscribes to) and processes event data from Kafka (e.g., `NotificationService`, `AnalyticsService`).
- **Broker:** An individual Kafka server instance inside a cluster responsible for storing and serving messages.
- **Kafka Cluster:** A set of interconnected Kafka brokers operating together for high availability, replication, and fault tolerance.
- **Topic:** A logical channel or category to which messages are published (e.g., `taaskr.booking.created`, `taaskr.payment.completed`).
- **Partition:** Each Kafka Topic is divided into multiple ordered segments called partitions distributed across brokers. Partitions enable horizontal scalability and parallel processing.
- **Partition Key:** A string value (e.g., `bookingId` or `userId`) attached to a message that determines which specific partition the message lands on via hashing (`hash(key) % numPartitions`). Messages with the same key ALWAYS go to the same partition.
- **Offset:** A unique, monotonically increasing sequential integer assigned to each record within a partition, serving as its immutable ID and position marker.
- **Consumer Group:** A group of consumer microservice instances working together to read from a topic. Kafka assigns each partition to exactly one consumer within a group, allowing load balancing.
- **Zookeeper / KRaft:** Zookeeper (legacy) or KRaft (modern Kafka Raft metadata mode) manages broker cluster metadata, controller election, and topic configurations.

2. Role of Kafka in the Taaskr Platform

Monolith State vs. Target Microservices Architecture

In the initial **Spring Boot Monolith** (`taaskr-backend`), event propagation happens in-memory within a single Java Virtual Machine (JVM) using Spring's `ApplicationEventPublisher`:

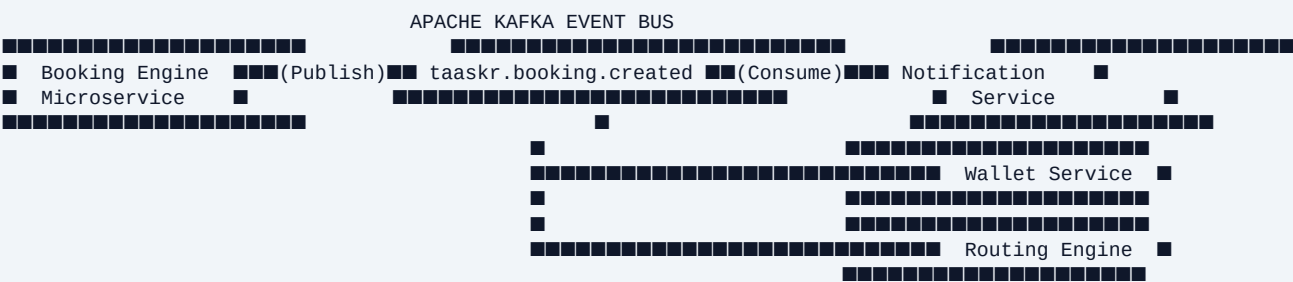


The Problem with Monolith In-Memory Events under High Load:

1. **Thread Blocking:** If SMS Gateway timeouts occur in `NotificationListener`, the customer's HTTP checkout thread is held open, exhausting Tomcat worker threads.
2. **Database Connection Starvation:** Long-running event listeners hold open HikariCP connections (`max-pool-size=5`), crashing the app.
3. **Cascading Failures:** A failure in an auxiliary listener (e.g. analytics logging) rolls back the entire database transaction for the core booking creation.

The Target Microservices Architecture with Apache Kafka

As detailed in the `MICROSERVICES_MIGRATION_BLUEPRINT.md`, Taaskr decomposes into 8 autonomous microservices. **Apache Kafka** forms the distributed, decoupled event spine:



How Kafka Transforms Taaskr:

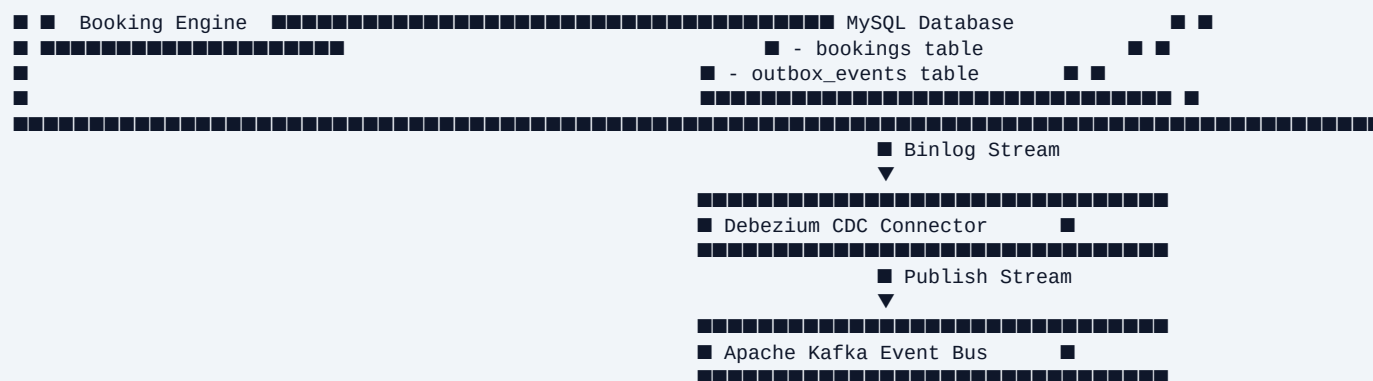
1. **Zero Thread Blocking:** `Booking Engine` commits the booking, emits `taaskr.booking.created` to Kafka, and immediately responds to the user in $< 50\text{ms}$.
2. **Independent Scalability:** If notification volume spikes during holiday promotions, you can scale `Notification Service` from 2 to 10 instances without modifying or redeploying `Booking Engine`.
3. **Fault Isolation & Backpressure Handling:** If `Notification Service` goes down for maintenance, Kafka buffers millions of messages safely on disk. When `Notification Service` recovers, it resumes processing from its last saved offset without losing a single message!

Dual-Write Prevention: Transactional Outbox Pattern & Debezium CDC

Directly writing to MySQL and publishing to Kafka inside a single method creates the dangerous **Dual-Write Vulnerability** (e.g. database commit succeeds, but Kafka broker network drops, leaving system in inconsistent state).

To guarantee **100% data consistency**, Taaskr utilizes the **Transactional Outbox Pattern** with **Debezium Change Data Capture (CDC)**:





1. `Booking Engine` writes to `bookings` table AND inserts a record into `outbox\_events` table within a SINGLE local MySQL ACID transaction.
2. Debezium CDC reads MySQL binary log (`binlog`) in real-time.
3. Debezium publishes the outbox message to Kafka topic `taaskr.booking.created`.

3. Step-by-Step Setup & Configuration Guide

1. Infrastructure Setup (`docker-compose.yml`)

Add Kafka and Zookeeper (or Kafka KRaft) containers to the Taaskr platform orchestration:

```
version: '3.8'
services:
  zookeeper:
    image: confluentinc/cp-zookeeper:7.5.0
    container_name: taaskr-zookeeper
    environment:
      ZOOKEEPER_CLIENT_PORT: 2181
      ZOOKEEPER_TICK_TIME: 2000
    ports:
      - "2181:2181"

  kafka:
    image: confluentinc/cp-kafka:7.5.0
    container_name: taaskr-kafka
    depends_on:
      - zookeeper
    ports:
      - "9092:9092"
      - "29092:29092"
    environment:
      KAFKA_BROKER_ID: 1
      KAFKA_ZOOKEEPER_CONNECT: zookeeper:2181
      KAFKA_ADVERTISED_LISTENERS: PLAINTEXT://kafka:9092,PLAINTEXT_HOST://localhost:29092
      KAFKA_LISTENER_SECURITY_PROTOCOL_MAP: PLAINTEXT:PLAINTEXT,PLAINTEXT_HOST:PLAINTEXT
      KAFKA_INTER_BROKER_LISTENER_NAME: PLAINTEXT
      KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR: 1
```

2. Spring Boot Dependencies (`pom.xml`)

```
<dependency>
  <groupId>org.springframework.kafka</groupId>
  <artifactId>spring-kafka</artifactId>
</dependency>
```

3. Application Properties (`application.yml`)

```
spring:
  kafka:
    bootstrap-servers: localhost:29092
    producer:
      key-serializer: org.apache.kafka.common.serialization.StringSerializer
      value-serializer: org.springframework.kafka.support.serializer.JsonSerializer
      acks: all
      retries: 3
    consumer:
      group-id: taaskr-notification-group
      auto-offset-reset: earliest
      enable-auto-commit: false
      key-deserializer: org.apache.kafka.common.serialization.StringDeserializer
      value-deserializer: org.springframework.kafka.support.serializer.JsonDeserializer
      properties:
        spring.json.trusted.packages: "com.taaskr.dto.event"
```

4. Java Implementation: Producer & Consumer

Event DTO Class

```
public record BookingCreatedEvent(
    Long bookingId,
    Long customerId,
    String customerEmail,
    String serviceName,
    Double amount,
    String city,
    String eventId
) {}
```

Kafka Producer Component

```
@Component
public class BookingEventProducer {
    private static final Logger log = LoggerFactory.getLogger(BookingEventProducer.class);
    private final KafkaTemplate<String, Object> kafkaTemplate;

    public BookingEventProducer(KafkaTemplate<String, Object> kafkaTemplate) {
        this.kafkaTemplate = kafkaTemplate;
    }

    public void publishBookingCreated(BookingCreatedEvent event) {
        String topic = "taaskr.booking.created";
        String key = String.valueOf(event.bookingId());

        kafkaTemplate.send(topic, key, event).whenComplete((result, ex) -> {
            if (ex == null) {
                log.info("Published event [{}] to topic [{}] partition [{}] at offset [{}]",
                    event.eventId(), topic, result.getRecordMetadata().partition(), result.getRecordMetadata().offset());
            } else {
                log.error("Failed to publish event [{}] to topic [{}]", event.eventId(), topic, ex);
            }
        });
    }
}
```

Idempotent Kafka Consumer Component

```
@Component
public class NotificationEventListener {
    private static final Logger log = LoggerFactory.getLogger(NotificationEventListener.class);
    private final StringTemplate redisTemplate;

    public NotificationEventListener(StringTemplate redisTemplate) {
        this.redisTemplate = redisTemplate;
    }
}
```

```

    @KafkaListener(topics = "taaskr.booking.created", groupId = "taaskr-notification-group")
    public void handleBookingCreated(@Payload BookingCreatedEvent event, Acknowledgment ack) {
        String redisIdempotencyKey = "kafka:idempotency:" + event.eventId();

        Boolean isFirstTime = redisTemplate.opsForValue().setIfAbsent(redisIdempotencyKey, "PROCESSED", Duration.ofDays(7));
        if (Boolean.FALSE.equals(isFirstTime)) {
            log.warn("Duplicate Kafka event detected [{}]. Skipping execution.", event.eventId());
            ack.acknowledge();
            return;
        }

        try {
            log.info("Dispatching SMS & Push Notification for Booking #{}", event.bookingId());
            // Notification logic here...
            ack.acknowledge(); // Manual commit after success
        } catch (Exception e) {
            log.error("Error processing notification event [{}]", event.eventId(), e);
            throw e; // Triggers Dead Letter Topic retry handler
        }
    }
}

```

4. Top Interview Questions & Detailed Technical Answers

Q1. What is the fundamental difference between Spring's `ApplicationEventPublisher` and Apache Kafka in Taaskr?

Answer: `ApplicationEventPublisher` provides synchronous or asynchronous event dispatching strictly confined within a single Java Virtual Machine (JVM). It cannot cross process boundaries.

In contrast, **Apache Kafka** is a distributed, persistent event broker that connects decoupled microservices running across different server nodes or Docker containers. Kafka stores messages immutably on disk, allows multiple independent consumer groups to read at their own pace, and handles high-throughput backpressure safely.

Q2. How do you solve the "Dual-Write" problem when writing to a database and publishing a Kafka event in Taaskr?

Answer: The dual-write problem occurs when a service attempts to write to a database and publish to Kafka sequentially (e.g. DB transaction commits, but network drops before Kafka publish succeeds, causing data divergence).

We solve this in Taaskr using the **Transactional Outbox Pattern** combined with **Debezium Change Data Capture (CDC)**:

1. The service writes the business record (e.g., `bookings`) and an event payload into an `outbox\_events` SQL table within the **same local database transaction**.
2. Debezium CDC monitors the MySQL binary log (`binlog`), captures the new outbox rows asynchronously, and streams them directly into Kafka.
3. This eliminates distributed two-phase commits (2PC) while guaranteeing 100% reliable event delivery.

Q3. How does Kafka ensure strict event ordering for a specific booking in Taaskr?

Answer: Kafka guarantees strict total ordering ONLY within a single topic partition.

To ensure that state transitions for a given booking (e.g., `BookingCreated` -> `BookingAssigned` -> `BookingStarted` -> `BookingCompleted`) are consumed in exact sequential order, we pass `bookingId` as the **Partition Key** when publishing records. Kafka's hash function (`hash(key) % numPartitions`) routes all events with the same `bookingId` to the **exact same partition**, where they are

appended sequentially and consumed in order by a single consumer thread.

Q4. What is the Idempotent Consumer Pattern, and why is it necessary with Kafka?

Answer: Kafka provides **At-Least-Once** delivery semantics by default. Under network blips or consumer rebalancing, a message might be delivered to a consumer service more than once.

To prevent duplicate actions (e.g. charging a wallet twice or sending duplicate SMS), consumer services implement the **Idempotent Consumer Pattern**:

1. Every event payload carries a unique UUID (`eventId`).
2. Before processing, the consumer checks Redis using `SETNX kafka:idempotency:<eventId> "PROCESSED" EX 604800`.
3. If Redis returns `false`, the message was already processed; the consumer logs a duplicate warning and commits the offset immediately without re-executing business logic.

Q5. What happens if a consumer service crashes or goes offline while processing Kafka messages?

Answer: Kafka tracks consumer progress using **Consumer Group Offsets**.

- If `NotificationService` crashes, its active consumer group partition assignment is revoked and assigned to another healthy consumer instance in the group (or held until the service restarts).
- Because `enable-auto-commit` is set to `false`, offsets are committed manually (`ack.acknowledge()`) ONLY after processing completes.
- When the service restarts, it queries Kafka for its last committed offset and resumes reading seamlessly from where it left off, avoiding message loss.

Q6. How does Kafka handle "Poison Pill" messages that repeatedly crash consumers?

Answer: A poison pill is a corrupted payload (e.g. malformed JSON or null fields) that repeatedly throws unhandled exceptions during deserialization or business execution, putting the consumer in an infinite retry loop.

In Taaskr, we configure a **Dead Letter Topic (DLT / DLQ)** handler using Spring Kafka's `DefaultErrorHandler` and `DeadLetterPublishingRecoverer`:

1. If a message fails after 3 retry attempts with exponential backoff (e.g. 1s, 2s, 4s), the error handler intercepts the message.
2. The failing message is routed to a Dead Letter Topic named `taaskr.booking.created.DLT`.
3. An administrative dashboard monitors the DLT topic, allowing developers to inspect corrupted payloads, patch bugs, and replay the messages manually.

Q7. How does Distributed Tracing work across Kafka asynchronous boundaries?

Answer: In HTTP REST, distributed tracing headers (`traceparent`) are passed via HTTP headers. Across Kafka, **Micrometer Tracing** (or Spring Cloud Sleuth) automatically injects the W3C `traceparent` context into **Kafka Message Headers**.

When the downstream microservice consumes the message, Micrometer extracts the `traceId` from the Kafka header, maintaining a single end-to-end trace span across Zipkin / Jaeger dashboards from the moment the user clicks "Book Now" through all asynchronous background events.

Q8. What are Kafka Consumer Groups and how does Partition Rebalancing work?

Answer: A **Consumer Group** is a set of consumer instances sharing the work of reading from a topic. If a topic has 4 partitions and a consumer group has 2 instances, Kafka assigns 2 partitions to each instance.

If a 3rd instance joins the group, Kafka triggers a **Rebalance**:

1. The Group Coordinator broker pauses partition consumption briefly.
2. Partitions are reassigned across the 3 instances (e.g. 2-1-1 distribution).
3. Modern Kafka uses Cooperative Sticky Assignors to reassign only necessary partitions without stopping healthy consumer threads completely.

Q9. What is the difference between `acks=0`, `acks=1`, and `acks=all` (or `-1`) in Kafka Producer?

Answer:

- `acks=0`: Producer sends message and doesn't wait for acknowledgment. Fastest, but highest risk of data loss.
- `acks=1`: Producer waits for the Leader broker to acknowledge writing to its local log. Fast, but risks data loss if Leader crashes before replicating to In-Sync Replicas (ISR).
- `acks=all` (or `-1`): Producer waits for the Leader AND all In-Sync Replicas (ISRs) to acknowledge writing to disk. **Used in Taaskr** for financial and booking events to guarantee zero data loss.

Q10. How do you monitor Kafka cluster health and Consumer Lag in production?

Answer: Consumer Lag is the difference between the latest offset written by producers to a partition and the offset currently committed by the consumer group.

- **Tools Used:** Prometheus + Grafana dashboard with `kafka_consumergroup_lag` metrics, plus Kafka UI / Confluent Control Center.
- **Alerting Threshold:** If consumer lag on `taaskr.booking.created` exceeds 1,000 messages or increases continuously for > 3 minutes, PagerDuty alerts fire to trigger auto-scaling of consumer pod replicas in Kubernetes.
